

Terraform Remote Backend

What is a Terraform Remote Backend?

A **Terraform backend** determines **how and where** Terraform stores its **state file** (terraform.tfstate). By default, Terraform uses a **local backend**, which stores the state file on your local machine.

A **remote backend**, on the other hand, stores the state file **remotely**—in a cloud service or remote server—allowing for:

- **Collaboration** across teams
- **State locking** to prevent concurrent changes
- **Security** and centralized management

Why Use a Remote Backend?

Here are the key reasons:

1. Team Collaboration

Multiple team members can work on the same infrastructure without overwriting each other's changes.

2. State Locking

Prevents race conditions. For example, AWS S3 with DynamoDB can lock the state during operations.

3. Security

Remote backends can be encrypted and access-controlled (e.g., using IAM roles in AWS).

4. Disaster Recovery

Remote storage is more resilient than local files. You can back up and restore easily.

5. Automation

Remote backends integrate well with CI/CD pipelines.

Deep Dive: How Remote Backends Work

◇ Common Remote Backends

- AWS S3 (with optional DynamoDB for locking)
- Terraform Cloud / Enterprise
- Azure Blob Storage
- Google Cloud Storage
- Consul

◇ Example: AWS S3 Remote Backend

```
terraform {  
  backend "s3" {  
    bucket      = "my-terraform-state-bucket"  
    key         = "env/dev/terraform.tfstate"  
    region      = "us-east-1"  
    dynamodb_table = "terraform-locks"  
    encrypt     = true  
  }  
}
```

 This configuration must be in your main.tf or backend.tf file, and initialized using terraform init.

☑ Why Use an S3 Bucket in Terraform Remote Backend?

☑ 1. Remote Storage of Terraform State

Terraform needs to store a file called `terraform.tfstate`, which keeps track of:

- What infrastructure has been created
- Resource metadata (like IDs, IPs, etc.)
- Dependencies between resources

By default, this file is stored **locally**, but that's not ideal for:

- Teams
- CI/CD pipelines
- Production environments

So, we use **Amazon S3** to store this file **remotely and securely**.

☑ 2. Centralized and Shared Access

Using S3 allows:

- Multiple team members to access the same state
- CI/CD tools to read/write the state
- Consistent infrastructure tracking across environments

☑ 3. Versioning and Recovery

S3 supports **versioning**, so you can:

- Roll back to a previous state file
- Recover from accidental changes or corruption

☑ 4. Security and Encryption

S3 supports:

- **Server-side encryption (SSE)**
- **IAM policies** to control access
- **Access logging** for auditing

☑ 5. Scalability and Availability

S3 is:

- Highly available (99.999999999% durability)
- Scalable to any size
- Integrated with AWS services

🔄 How It Works Together

Here's a simple flow:

1. You run terraform apply
2. Terraform checks the state file in the S3 bucket
3. It locks the state using DynamoDB (if configured)
4. It updates the infrastructure
5. It writes the new state back to S3
6. It releases the lock in DynamoDB

🔒 Why Use dynamodb_table?

Terraform needs to **lock the state file** during operations like plan and apply to prevent **race conditions**—where two people or processes try to change the infrastructure at the same time.

Without locking, you could end up with:

- **Corrupted state files**
- **Unpredictable infrastructure changes**
- **Hard-to-debug errors**

🏠 What is DynamoDB Doing Here?

```
dynamodb_table = "terraform-locks"
```

Terraform uses the **DynamoDB table** to:

- **Create a lock** before modifying the state

- **Release the lock** after the operation is complete
- **Prevent others** from running Terraform at the same time

🔧 How to Set Up the DynamoDB Table

You need to create a DynamoDB table with:

- **Table name:** terraform-locks (or any name you choose)
- **Partition key:** LockID (type: String)

Here's a Terraform snippet to create it:

```
resource "aws_dynamodb_table" "terraform_locks" {  
  
    name      = "terraform-locks"  
  
    billing_mode = "PAY_PER_REQUEST"  
  
    hash_key   = "LockID"  
  
    attribute {  
  
        name = "LockID"  
  
        type = "S"  
  
    }  
  
}
```

📄 Summary

Feature	Purpose
dynamodb table	Enables state locking
Prevents	Concurrent terraform apply or plan
Required for	Safe collaboration in teams
Works with	S3 backend only (not required for Terraform Cloud)

Setup Steps

Step 1: Create S3 Bucket and DynamoDB Table

You can do this manually or via Terraform or CLI

1. S3 Bucket – for storing the Terraform state file

This is where Terraform will store the terraform.tfstate file remotely.

```
resource "aws_s3_bucket" "terraform_state" {  
    bucket = "my-terraform-state-bucket"  
  
    versioning {  
        enabled = true  
    }  
  
    server_side_encryption_configuration {  
        rule {  
            apply_server_side_encryption_by_default {  
                sse_algorithm = "AES256"  
            }  
        }  
    }  
}  
  
resource "aws_s3_bucket_policy" "terraform_state_policy" {  
    bucket = aws_s3_bucket.terraform_state.id  
  
    policy = jsonencode({  
        Version = "2012-10-17"  
  
        Statement = [  

```

```

{
  Sid      = "TerraformStateAccess"
  Effect   = "Allow"
  Principal = {
    AWS = "arn:aws:iam::123456789012:role/TerraformRole"
  }
  Action = [
    "s3:GetObject",
    "s3:PutObject",
    "s3:DeleteObject",
    "s3:ListBucket"
  ]
  Resource = [
    "arn:aws:s3:::my-terraform-state-bucket",
    "arn:aws:s3:::my-terraform-state-bucket/*"
  ]
}
]
})
}

```

Replace This:

- "arn:aws:iam::123456789012:role/TerraformRole" → with your actual IAM role or user ARN.
- "my-terraform-state-bucket" → with your actual bucket name if different.

2. DynamoDB Table – for state locking

This prevents multiple users from running terraform apply at the same time.

```
resource "aws_dynamodb_table" "terraform_locks" {  
    name      = "terraform-locks"  
    billing_mode = "PAY_PER_REQUEST"  
    hash_key   = "LockID"  
    attribute {  
        name = "LockID"  
        type = "S"  
    }  
}
```

Terraform Code to Create IAM Role for Terraform

```
resource "aws_iam_role" "terraform_role" {  
    name = "TerraformRole"  
    assume_role_policy = jsonencode({  
        Version = "2012-10-17",  
        Statement = [  
            {  
                Effect = "Allow",  
                Principal = {  
                    Service = "ec2.amazonaws.com" # or "lambda.amazonaws.com" or another service  
if needed  
                },  
                Action = "sts:AssumeRole"  
            }  
        ]  
    })  
}
```



```

    }
  ]
})
}

resource "aws_iam_policy" "terraform_backend_policy" {

  name      = "TerraformBackendAccess"

  description = "Policy to allow Terraform to access S3 and DynamoDB for remote
backend"

  policy = jsonencode({

    Version = "2012-10-17",

    Statement = [

      {

        Effect = "Allow",

        Action = [

          "s3:GetObject",

          "s3:PutObject",

          "s3:DeleteObject",

          "s3:ListBucket"

        ],

        Resource = [

          "arn:aws:s3:::my-terraform-state-bucket",

          "arn:aws:s3:::my-terraform-state-bucket/*"

        ]

      },

      {

        Effect = "Allow",

```

```

    Action = [
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb>DeleteItem",
        "dynamodb:Scan",
        "dynamodb:Query",
        "dynamodb:UpdateItem"
    ],
    Resource = "arn:aws:dynamodb:ap-south-1:123456789012:table/terraform-locks"
}

I
})
}

resource "aws_iam_role_policy_attachment" "attach_policy" {
    role      = aws_iam_role.terraform_role.name
    policy_arn = aws_iam_policy.terraform_backend_policy.arn
}

```

Replace These Values:

- "my-terraform-state-bucket" → your actual S3 bucket name
- "terraform-locks" → your DynamoDB table name
- "123456789012" → your AWS account ID
- "ap-south-1" → your AWS region

☒ Step 2: Configure Backend in Terraform

```
terraform {  
  backend "s3" {  
    bucket      = "my-terraform-state-bucket"  
    key         = "env/dev/terraform.tfstate"  
    region      = "ap-south-1"  
    dynamodb_table = "terraform-locks"  
    encrypt     = true  
  }  
}
```

Multi-Environment Architecture

If Use workspaces or separate backends for:

- dev, staging, prod
- Isolated state files per environment
- CI/CD pipelines that deploy to different environments

```
key = "env/${terraform.workspace}/terraform.tfstate"
```

☒ Step 3: Initialize Terraform

```
terraform init
```

Terraform will prompt you to migrate your state to the remote backend.

☒ Step 4: Use Terraform Normally

Run terraform plan, apply, etc. Terraform will now use the remote state.

Advanced Concepts

State Encryption

Ensure your backend supports encryption (e.g., S3 with SSE).

State Versioning

Enable versioning in S3 to track changes and roll back if needed.

Workspaces

Use Terraform workspaces to manage multiple environments (e.g., dev, staging, prod) with the same backend.

Backend Configuration Separation

Avoid hardcoding sensitive values. Use partial configuration and pass values via CLI or environment variables.

Best Practices

- **Never commit .tfstate files to Git**
- Use **remote backends** for production environments
- Enable **locking and versioning**
- Use **workspaces** for environment separation
- Use **Terraform Cloud** for built-in collaboration and security

Destroy the Remote Backend (S3 + DynamoDB) and Remote State File

Terraform remote backend (S3 + DynamoDB) is currently in use and you're planning to **destroy the entire infrastructure**, including the **remote state** .

☑ Step 1: Migrate State Back to Local (Optional but Recommended)

This step ensures you have a backup of the state before deletion.

```
terraform init -migrate-state
```

This will move the state from S3 to your local machine (*terraform.tfstate*).

☑ Step 2: Remove the Backend Block from Your Code

Comment out or delete this block:

```
terraform {  
  backend "s3" {  
    bucket      = "your-bucket-name"  
    key         = "your/key/path.tfstate"  
    region      = "your-region"  
    dynamodb_table = "your-lock-table"  
    encrypt     = true  
  }  
}
```

☑ Step 3: Reinitialize Terraform

```
terraform init
```

This will now use the **local backend** and detach from the remote one.

☑ Step 4: Destroy the Remote Backend Resources

If you created the S3 bucket and DynamoDB table using Terraform, just run:

```
terraform destroy
```

You can also manually empty the bucket using the AWS Console or CLI.

☑ Step 5: Verify Remote State is Gone

- Check that the **S3 bucket is deleted**
- Check that the **DynamoDB table is deleted**
- Confirm that no *.tfstate* file remains in S3